



UD3

INTRODUCCIÓN A JAVASCRIPT

3.6 JS ORIENTADO A OBJETOS

MP_0373

Lenguaje de marcas y
Sistemas de gestión de
la información

Introducción

La programación orientada a objetos es una manera de programar, organizando el código en bloques denominados clases, y mediante la creación de objetos (instancias de dichas clases) que comunicándose entre sí crean los programas.

Las clases de la programación orientada a objetos son cercanas a cómo se expresarían los conceptos en el mundo real, pero, por así decirlo, codificados para que sea un programa quién los cree, modifique o destruya. Esta manera de abstraer bloques de código cercanos al mundo real hace que las aplicaciones sean más fáciles de entender, mantener y de hacer el código más reutilizable.

¿Qué es una clase? ¿Y un objeto?

Una clase es un concepto con unas propiedades definidas y unos métodos que implementan sus funcionalidades. Por ejemplo, pensando en el mundo real, un coche podría codificarse como una clase, Un coche tiene:

- ⊕ **Propiedades:** Como el color, marca, tiene o no airbag, abs, matrícula, etc.
- ⊕ **Métodos:** El coche puede arrancar, detenerse, acelerar, frenar, etc.

Al pensar en esta clase abstracta de coche, se puede pensar en todos los coches del mundo: todos los coches tienen esas propiedades y esos métodos. La **idea de coche es la clase** y la representación real de esa idea es un objeto: un coche verde, matrícula XXXX de la marca Seat. Un objeto es una **instancia** de una clase.

Document ready

De nada sirve un objeto por sí mismo, al igual que en la vida real, en los programas, los objetos tienen que hablar entre sí para solventar problemas.

Siguiendo el ejemplo del mundo real, un coche pertenece a una persona, se guarda en un garaje o circula por una autopista. Persona, garaje o autopista también se pueden codificar como clases, de manera que tienen, albergan o circulan por ellas instancias de la clase *coche*. De esta manera se crean los programas, se codifica el modelo mental de la realidad para que la programación sea más cercana a ella y más fácil de entender por los desarrolladores.

Definición de una clase en JavaScript

Para definir una clase en JavaScript se utiliza la palabra reservada `class` seguida del nombre de la clase. Los nombres de clase como los de variables son case-sensitive pero con convención, se empiezan con mayúscula.

```
class Coche {  
}
```

Dentro de las llaves `{}` irá todo el código de la clase y de esa manera queda encapsulado: todo lo que tiene que ver con la naturaleza de la clase, está ahí dentro.

Las clases tienen un método especial llamado constructor que se ejecuta automáticamente al instanciar una clase. Para instanciar una clase se utiliza la palabra reservada `new` seguida del nombre de la clase y de unos paréntesis `()`.

```
class Coche {  
  constructor() {  
    console.log("Se ha creado un coche");  
  }  
}
```

```
// Se instancia la clase coche y se deja el objeto en la variable coche  
const coche = new Coche();  
// Mostrará por consola:  
// Se ha creado un coche
```

Propiedades

La palabra reservada `this` utilizada dentro de una clase se refiere a la instancia de la misma clase, es decir, esa parte de memoria donde se guarda toda la información sobre la instancia.

`this` en JavaScript es utilizado para guardar propiedades de una clase. Una propiedad de una clase se asemeja a una variable: un valor accesible mediante un nombre de propiedad que se guarda en la instancia. Por ejemplo, para una clase `Coche` una propiedad sería el color.

Es habitual definir parte de estas propiedades en el constructor, pero se pueden modificar o acceder a ellas en cualquier momento, tanto dentro como fuera de la clase utilizando la misma notación que para acceder a claves de objetos (con `.` o con `[clave]`).

```
class Coche {
  constructor(color, marca) {
    this.color = color;
    this.marca = marca;
  }
}

const seatRojo = new Coche('rojo', 'seat');
console.log(seatRojo.color); // rojo

// Sobreescribiendo propiedad
seatRojo.color = 'verde';
console.log(seatRojo.color); // verde

// Añadiendo nueva propiedad
console.log(seatRojo.nPuertas); // undefined
seatRojo.nPuertas = 4;
console.log(seatRojo.nPuertas); // 4
```

Métodos

Los métodos de una clase se asemejan a funciones, es decir, bloques de código que ejecutan un determinado número de sentencias. Al igual que el concepto de clase, un método también se asemeja al modelo mental de qué cosas puede hacer una clase.

```
class Coche {
  constructor() {
    this.encendido = false;
  }

  arrancar() {
    this.encendido = true;
  }
}
```

```
    detener() {  
        this.encendido = false;  
    }  
}
```

En el ejemplo anterior, se han añadido los métodos arrancar y detener, gracias a ellos se puede manejar la propiedad encendida. De esta manera la clase *Coche* es más sencilla de utilizar y recrea un poco más lo que un coche hace en la vida real.

Los métodos pueden ser tan complicados como se necesite, no dejan de ser funciones dentro de la clase y que pueden acceder al espacio en memoria que ocupa la instancia a través de la palabra *this*.

Un ejemplo un poco más completo:

```
class Coche {  
    constructor(marca, color, bastidor) {  
        this.marca = marca;  
        this.color = color;  
        this.bastidor = bastidor;  
        this.encendido = false;  
        this.velocidad = 0;  
    }  
  
    arrancar() {  
        if (!this.encendido) {  
            this.encendido = true;  
        }  
    }  
  
    apagar() {  
        if (this.encendido) {  
            this.encendido = false;  
        }  
    }  
  
    acelerar(velocidad) {
```

```

    this.velocidad += velocidad;
  }

  frenar(velocidad) {
    this.velocidad -= velocidad;
    if (this.velocidad < 0) {
      // No puede ir a velocidad negativa
      this.velocidad = 0;
    }
  }
}

itv() {
  if (this.velocidad > 0) {
    console.log("detenga el coche");
  } else if (this.encendido) {
    console.log("apague el coche");
  } else {
    console.log('Marca: ', this.marca);
    console.log('Color: ', this.color);
    console.log('Bastidor: ', this.bastidor);
  }
}
}

```

Conceptos avanzados de POO en Javascript

Encapsulación

La encapsulación es un mecanismo de la programación orientada a objetos. En una clase puede haber métodos y/o propiedades de una instancia que el programador no quiere que puedan utilizarse directamente, haciendo así restringido su acceso.

```

class Coche {
  constructor(bastidor) {
    this.bastidor = bastidor;
  }
}

const coche = new Coche('123');

```

```
coche.bastidor = 456;  
console.log(coche.bastidor); // 456
```

En el ejemplo anterior, el número de bastidor del coche que se define en el constructor se puede cambiar de una manera muy sencilla. El número de bastidor en el mundo real es algo relacionado intrínsecamente con el coche y no se puede cambiar. Este comportamiento se puede codificar con una propiedad privada, para ello, simplemente se define dentro del cuerpo de la clase el nombre de la propiedad con un # como prefijo:

```
class Coche {  
  #bastidor;  
  constructor(bastidor) {  
    this.#bastidor = bastidor;  
  }  
  
  leerBastidor() {  
    return this.#bastidor;  
  }  
}  
  
const coche = new Coche('123');  
console.log(coche.bastidor); // undefined  
console.log(coche.leerBastidor()); // 123
```

Un método también puede ser privado, simplemente hay que añadirle un # como prefijo:

```
class Coche {  
  constructor() {  
    #procesosInternos();  
  }  
  
  #procesosInternos() {  
    ...  
  }  
}
```

En el anterior ejemplo, hay procesos internos que ocurren al crearse un coche que solo se deben ejecutar en el constructor, por ello, se crean dentro de un método privado para que no puedan llamarse desde fuera.

Relacionado con la encapsulación, JavaScript dispone de getters y setters que es una manera de hacer propiedades virtuales, es decir, desde fuera parece una propiedad, pero dentro de la clase no lo es:

```
class Persona {
  #nombre
  #apellidos

  constructor(nombre, apellidos) {
    this.#nombre = nombre;
    this.#apellidos = apellidos;
  }

  get nombreCompleto() {
    return this.#nombre + ' ' + this.#apellidos;
  }

  set nombreCompleto(input) {
    const arrayInput = input.split(' '); // Devuelve un array con cada
    elemento que esté separado por el carácter ' ' (espacio)
    this.#nombre = arrayInput[0];
    this.#apellidos = arrayInput[1];
  }
}

const persona = new Persona('Alonso', 'Paz');

console.log(persona.nombreCompleto); // Alonso Paz
persona.nombreCompleto = "Martin Sola";
console.log(persona.nombreCompleto); // Martin Sola
```


Herencia

Otro concepto relacionado con la programación orientada a objetos es la herencia, un mecanismo mediante el cual una clase hija hereda código de una clase padre.

Siguiendo con la codificación del modelo mental del mundo real y pensando en una buena estructura de código y reutilización de este, se puede pensar en clases que de alguna manera sean parecidas y se pueda encontrar un padre de estas. Poniendo un ejemplo en la vida real: un sedán, una furgoneta y un descapotable con conceptos diferentes, pero todos heredan del concepto coche: Todos tienen 4 ruedas, un volante o necesitan un conductor.

Es por lo tanto que se puede codificar una clase padre que contenga código que las clases hijas puedan utilizar y de esta manera las clases hijas también tienen algo de la clase padre, es decir, son en parte también padre.

```
class Coche {  
  constructor(bastidor) {  
    this.bastidor = bastidor;  
    this.encendido = false;  
  }  
  
  arrancar() {  
    this.encendido = true;  
  }  
  
  apagar() {  
    this.encendido = false;  
  }  
}  
  
class Sedan extends Coche {  
  constructor(bastidor) {  
    super(bastidor);  
  }  
}  
  
class Descapotable extends Coche {  
  constructor(bastidor) {
```

```
    super(bastidor);
    this.capotaAbierta = false;
}

capotar() {
    this.capotaAbierta = false;
}

descapotar() {
    this.capotaAbierta = true;
}
}

const sedan = new Sedan('123');
console.log(sedan.bastidor); // 123
sedan.arrancar();
sedan.apagar();
console.log(sedan.encendido); // false

const descapotable = new Descapotable('456');
descapotable.descapotar();
console.log(descapotable.capotaAbierta); // true
```

La codificación de clases hijas en JavaScript se hace añadiendo la palabra reservada `extend` seguida del nombre de la clase a la que extiende en la definición de la clase.

Además, es necesario llamar al constructor del padre a través del constructor del hijo, de no ser así, el padre no se construye y la clase no tendrá contexto de ejecución (no tendrá `this`). Para llamar al padre a través del hijo se utiliza la palabra `super` y para construir al padre, simplemente ejecutarlo: `super ()`.

```
class Coche {  
  constructor(marca, color, bastidor) {  
    this.marca = marca;  
    this.color = color;  
    this.bastidor = bastidor;  
    this.encendido = false;  
    this.velocidad = 0;  
  }  
  
  arrancar() {  
    if (!this.encendido) {  
      this.encendido = true;  
    }  
  }  
  
  apagar() {  
    if (this.encendido) {  
      this.encendido = false;  
    }  
  }  
  
  acelerar(velocidad) {  
    this.velocidad += velocidad;  
  }  
  
  frenar(velocidad) {  
    this.velocidad -= velocidad;  
    if (this.velocidad < 0) {  
      // No puede ir a velocidad negativa  
      this.velocidad = 0;  
    }  
  }  
  
  itv() {  
    if (this.velocidad > 0) {  
      console.log("detenga el coche");  
    } else if (this.encendido) {  
      console.log("apague el coche");  
    }  
  }  
}
```

```
    } else {  
      console.log('Marca: ', this.marca);  
      console.log('Color: ', this.color);  
      console.log('Bastidor: ', this.bastidor);  
    }  
  }  
}
```